

Performance Analysis of a Tiny Diffusion Model Implementation

A Comparative Profiling Study

Johann Bonneau

Julien Royer

June 9, 2026

Abstract

Diffusion models are powerful generative models but are often associated with high computational costs due to their iterative structure. This report presents a performance analysis of *tiny-diffusion*¹, a minimal PyTorch implementation of probabilistic diffusion models operating on low-dimensional 2D datasets. By combining Python-level and system-level profiling (cProfile, Hyperfine) with framework-level profiling (PyTorch Profiler), we identify computational bottlenecks during training. The goal is to characterize the performance structure of diffusion models in a controlled setting and to extract actionable optimization insights.

1 Introduction

Diffusion-based generative models rely on an iterative denoising process that requires repeated evaluations of a neural network. In large-scale applications such as image or audio generation, this structure leads to significant computational costs, often obscured by architectural complexity.

In this work, we deliberately focus on a minimal implementation, named *tiny-diffusion*, designed for pedagogical clarity and experimental simplicity. By operating on low-dimensional 2D datasets and using a simple MLP-based model, this implementation provides a controlled environment for detailed performance analysis.

The objective of this report is not to improve generative quality, but to analyze the **computational behavior** of the *tiny diffusion* model implementation. Specifically, we aim to:

- characterize the runtime structure of training and sampling,
- identify performance bottlenecks at multiple abstraction levels,
- compare complementary profiling tools,
- derive optimization insights grounded in empirical measurements.

2 System Under Study

This section describes the system benchmarked and optimized in the remainder of the report. It provides the minimal context needed to interpret Sections 3–5: what the model computes and how the codebase executes training (including where overhead can be introduced).

¹Fork of the Tiny diffusion with benchmark scripts <https://github.com/FAR3L/tiny-diffusion>.

2.1 Model overview (Tiny Diffusion denoiser)

Our implementation targets a minimal diffusion-model training setup on 2D point datasets. The denoiser is an MLP that takes as input a noisy point $x_t \in \mathbb{R}^2$ and a discrete timestep $t \in \{0, \dots, T-1\}$. Each coordinate and the timestep are mapped to positional embeddings, concatenated, and processed by a residual MLP. The model outputs a 2D vector $\hat{\epsilon} \in \mathbb{R}^2$ corresponding to the predicted noise component at the current timestep. During training, batches of clean samples x_0 are corrupted by sampling Gaussian noise ϵ and generating x_t using a noise scheduler; the denoiser is trained with an MSE loss between $\hat{\epsilon}$ and ϵ .

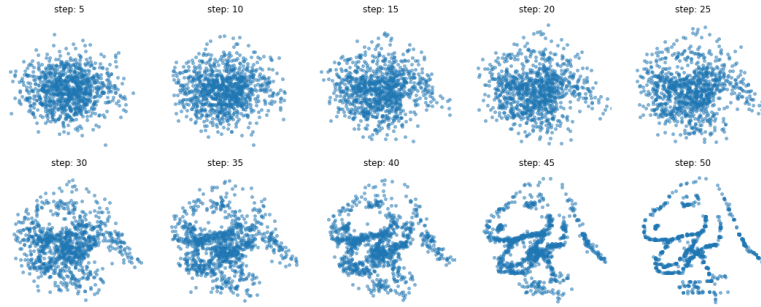


Figure 1: Denoising (reverse diffusion): starting from Gaussian noise, the model is applied iteratively across timesteps to recover samples from the learned data distribution.

2.2 Code Structure and Execution Flow

The core DDPM logic of this project lives in `DDPM.py`: it contains the training loop, the optional sampling routine, and the glue code that connects the model with the noise scheduler and the optimizer. The remaining modules mostly serve supporting roles (dataset generation/loading, utilities for embeddings, plotting, and profiling), so in what follows we focus on the *execution flow* that determines both learning behavior and runtime characteristics.

Figure 2 summarizes this flow. After a short initialization phase (argument parsing and component instantiation), execution enters the *training loop*, which is the dominant workload throughout the run. Each iteration draws a minibatch of clean points x_0 and trains the MLP to predict the injected Gaussian noise: a timestep t is sampled, a noisy point x_t is constructed using the scheduler’s forward diffusion rule, the model predicts $\hat{\epsilon} = \text{MLP}(x_t, t)$, and the parameters are updated by minimizing $\text{MSE}(\epsilon, \hat{\epsilon})$. The optional sampling block (controlled by a flag and typically triggered at a fixed interval) does not replace training; instead it provides a periodic qualitative check by starting from pure noise $x_T \sim \mathcal{N}(0, I)$ and iterating the scheduler’s reverse step down to a generated x_0 . Importantly, whether sampling is executed or skipped, the control flow rejoins at the output stage and then continues to the next iteration, which is exactly what the loop-back arrow in Figure 2 is meant to make explicit.

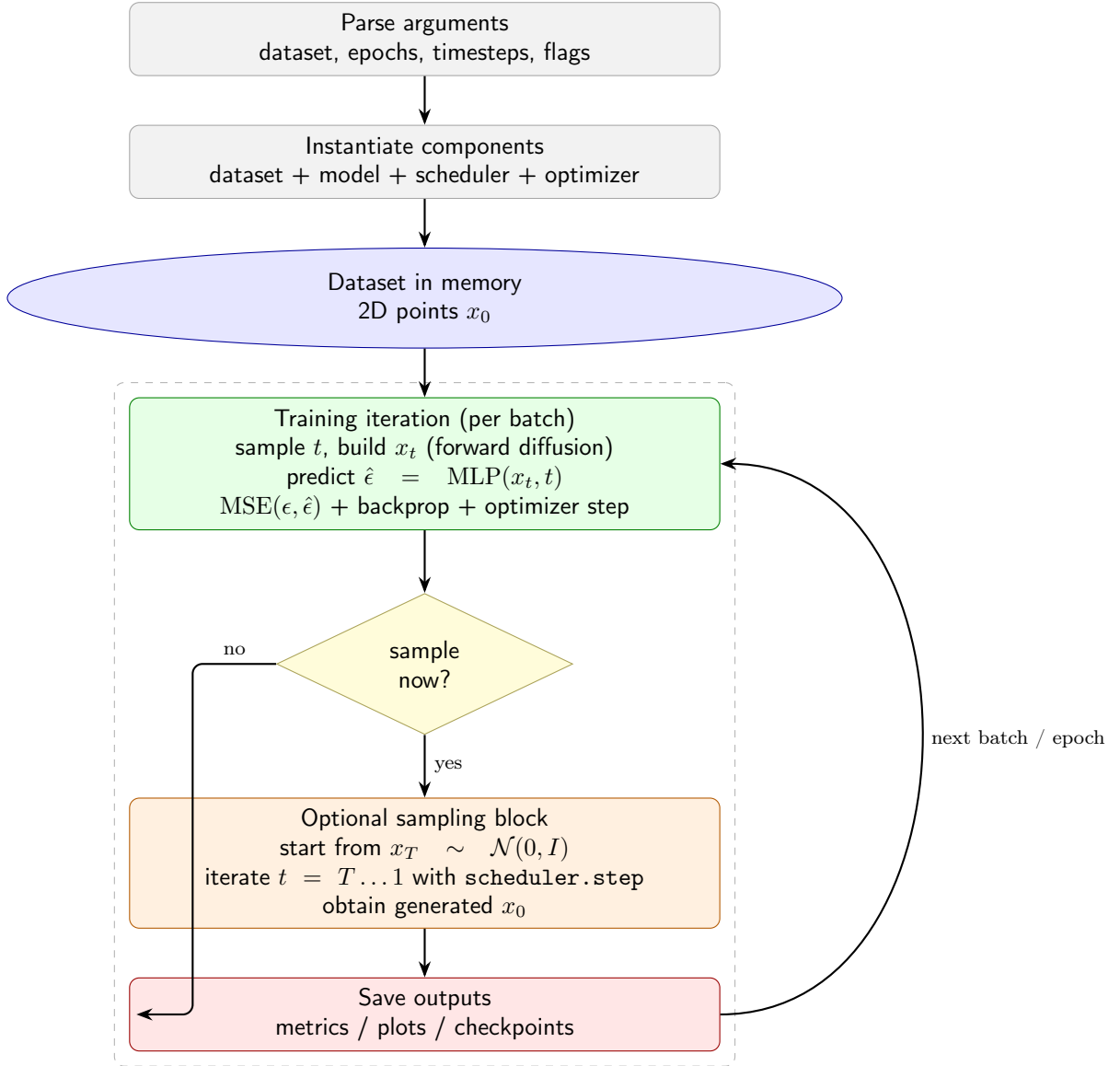


Figure 2: DDPM execution flow in DDPM.py: gray denotes setup, blue denotes data, green denotes the per-batch training step, orange denotes the optional sampling routine, and red denotes output/logging. The loop continues regardless of whether sampling is executed.

3 Profiling Methodology

This section summarizes the profiling tools used in our optimization workflow and how each tool is applied to the `ddpm.py` training workload. We combine system-level benchmarking for robust end-to-end runtime measurement and Python-level profiling to localize interpreter and orchestration overhead. Framework-level profiling (PyTorch Profiler) is planned but not yet integrated into the reporting pipeline.

3.1 Hyperfine (System-level benchmarking)

Hyperfine measures end-to-end wall-clock execution time by repeatedly running a full command, capturing both computation and any runtime overhead (startup, Python orchestration, progress

display, etc.) as observed externally. Because it executes the complete program multiple times, it provides statistically meaningful estimates of typical runtime and variability. All Hyperfine benchmarks use the same invocation settings:

- **Repetitions:** `-r 5`
- **Warmup runs:** `-warmup 2`
- **Time unit:** `-time-unit second`

Hyperfine returns summary timing statistics over the repeated executions, including (at minimum) the mean runtime, standard deviation, median, minimum and maximum runtime, and the list of measured run times.

3.2 cProfile (Python-level profiling)

`cProfile` measures function-level execution inside the Python interpreter. It attributes time to Python function calls (including call counts) and reports both self time (time spent in the function body) and cumulative time (time including callees). This makes it suitable for identifying Python-side hotspots such as excessive call overhead, iterator costs, and high-level training-loop orchestration. We run `cProfile` through the standard module entrypoint: `python -m cProfile -o <stats> <command>`. For interpretation, we generate a top-functions summary by sorting by `tottime` (self time) and reporting the top 50 entries.

3.3 Memory profiling (RSS via `memory_profiler`)

We use memory profiling to estimate the process-level memory footprint of our workloads. The reported metric is **RSS (resident set size)** in MiB, i.e., the amount of physical memory currently used by the process. We summarize memory behavior using **peak RSS** over the profiled region and an **RSS delta** that captures the maximum observed increase during execution (a proxy for transient allocation pressure). In our current setup, memory profiling is applied to the **training loop only** (the main training function), so the reported values characterize memory usage during training rather than during standalone sampling or other auxiliary phases.

3.4 PyTorch Profiler (Framework-level profiling)

While our previous profiling approaches (memory profiling, Hyperfine, and `cProfile`) focused on identifying computational bottlenecks and memory usage patterns in the code implementation, we employed PyTorch Profiler for a complementary purpose: systematic hyperparameter benchmarking. This profiling methodology addresses a fundamentally different question—not *where* the code spends time, but rather *which configuration* yields the optimal balance between training efficiency and model performance. For our DDPM implementation, we leveraged this capability to conduct a comprehensive hyperparameter search across five key dimensions:

Table 1: Explored Hyperparameter Space

Hyperparameter	Tested Values
Batch Size	32, 64, 128, 256
Hidden Layer Size	128, 256
Number of Hidden Layers	3, 4
Embedding Dimensions	64, 128
Diffusion Timesteps	50, 100
Total Configurations	64

This resulted in 64 unique configurations, each trained for 20 epochs on the Dino dataset.

4 Baseline Benchmark

Our baseline benchmark is the reference measurement used to quantify all subsequent optimizations. It focuses on the **training workload only**. We explicitly remove (i) the sampling loop and (ii) progress tracking (`tqdm`) from the baseline measurement, because both were mainly used for **evaluation/visual checks** and introduce additional control-flow and monitoring overhead that makes training performance changes harder to interpret.

4.1 Baseline Workloads

We define two baseline workloads that share the same model and training configuration (same dataset and hyperparameters), but differ in how much orchestration overhead they include:

- **core_train**: minimal-overhead training reference (progress disabled; sampling avoided by using a large evaluation interval).
- **full_train**: end-to-end training entrypoint with standard runtime orchestration, while keeping export disabled to avoid I/O bias.

4.2 Baseline Timing Results (Hyperfine)

Table 2 reports the baseline wall-clock timing statistics (in seconds) and the additional time observed for the full workload relative to the core baseline.

Table 2: Baseline wall-clock timing statistics (seconds) and relative overhead.

Workload	Mean (s)	Std (s)	Relative overhead vs. core
core_train	13.665	0.083	—
full_train	18.896	0.101	38.3%

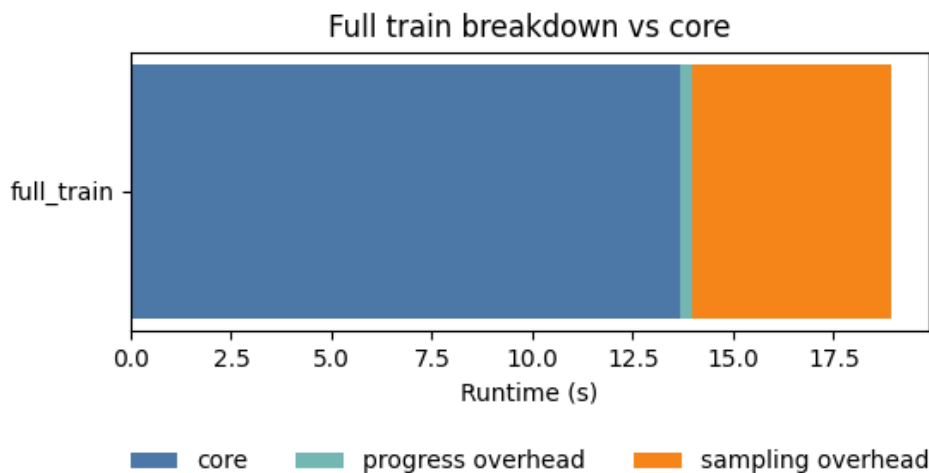


Figure 3: Overhead relative to `core_train` (baseline workloads visible among variants).

4.3 Baseline Python Hotspots (cProfile)

Tables 3 and 4 report the most expensive Python-level functions (by self time). In Table 4, the **highlighted lines** are functions that appear among the top costs in `full_train` but not in `core_train`, making them good indicators of overhead introduced by the full pipeline. Concretely, `torch._C._nn.linear` corresponds to the MLP’s linear layers and is present in both workloads, but it ranks higher in `full_train` because the model forward pass is invoked more often (e.g., when evaluation logic such as sampling is triggered), increasing the number of calls. The other highlighted entries, `torch.cat` and `positional_embeddings.forward`, correspond respectively to concatenations in the model input pipeline (e.g., combining embeddings) and to the computation of positional embeddings for inputs and timesteps; they become more prominent in `full_train` as these additional forward passes amplify their contribution.

Table 3: Top Python functions for `core_train` (cProfile, by self time).

Function	Total	Cumulative	Calls
<code>run_backward</code>	2.571	2.571	6,200
<code>dataset:<genexpr></code>	1.044	1.044	3,174,402
<code>torch._C._nn.linear</code>	0.960	0.960	31,500
<code>Tensor.mul_</code>	0.717	0.717	124,000
<code>dataset:__getitem__</code>	0.594	1.639	1,587,201

Table 4: Top Python functions for `full_train` (cProfile, by self time). Highlighted entries are overhead candidates.

Function	Total	Cumulative	Calls
<code>run_backward</code>	2.611	2.611	6,200
<code>torch._C._nn.linear</code>	2.372	2.372	81,000
<code>torch.cat</code>	1.065	1.065	64,800
<code>dataset:<genexpr></code>	1.038	1.038	3,174,402
<code>positional_embeddings.forward</code>	0.984	2.578	48,600

4.4 Baseline Memory Profiling (RSS)

Table 5 reports resident set size (RSS) statistics measured during the baseline workloads. **Peak RSS** is the maximum observed process memory usage during the run, while **Delta RSS** captures the maximum observed increase over the profiled execution window (an indicator of transient allocations and memory pressure during training). In our baseline, `full_train` is only marginally higher than `core_train` (+0.4 MiB peak RSS and +1.5 MiB delta RSS), which is insignificant relative to the overall footprint (~ 768 MiB) and may fall within normal runtime variability.

Table 5: Baseline memory usage (RSS) for core vs. full workloads.

Workload	Peak RSS (MiB)	Delta RSS (MiB)
<code>core_train</code>	768.0	19.0
<code>full_train</code>	768.4	20.5
Δ (full – core)	+0.4	+1.5

5 Data Optimization: Reduced Input-Pipeline Overhead

To reduce non-compute overhead inside the training loop, we modified the input pipeline so that batches are drawn directly from the in-memory tensors backing the dataset, instead of relying on per-batch `DataLoader` iteration and repeated dataset indexing. In our codebase, the training data for `dino` is already returned as a `torch.utils.data.TensorDataset`² and selected via `get_dataset("dino")`³; the baseline training path consumes it through a standard `DataLoader`⁴. The optimized path bypasses this iterator/indexing machinery and forms batches by slicing the underlying tensors (with equivalent batching and shuffling intent), removing high-frequency `__getitem__`/generator activity from the hot path and leaving forward diffusion, model forward, loss, backward, and optimizer step as the dominant per-iteration work.

5.1 Runtime results

Table 6 reports Hyperfine wall-clock results (seconds) for the baseline training workload and the data-optimized variant. The optimization reduces mean runtime from **13.67 s** to **11.36 s**, corresponding to a **16.9% time gain**.

Table 6: Runtime impact of the data-path optimization (Hyperfine, seconds).

Workload	Mean (s)	Std (s)	Time gain vs. baseline
core_train	13.665	0.083	–
core_plus_premat	11.364	0.110	16.9%

5.2 Memory results (RSS)

Table 7 reports RSS-based memory metrics for the baseline and the optimized input pipeline. The optimization introduces a small memory trade-off: peak RSS and RSS delta both increase slightly.

Table 7: Memory impact of the data-path optimization (RSS).

Workload	Peak RSS (MiB)	Delta RSS (MiB)
core_train	768.0	19.0
core_plus_premat	768.9	19.5
Δ (opt – baseline)	+0.9	+0.5

5.3 Trade-offs and limitations

This optimization improves runtime by reducing Python-side input overhead, but it comes with two practical trade-offs. First, it slightly increases memory pressure by keeping the dataset explicitly materialized and actively accessed as contiguous tensors; in our run this appears as a small increase in RSS (Table 7). Second, it reduces generality and scalability: the approach assumes the dataset fits in RAM and bypasses the standard `DataLoader` pipeline, which is typically required for large or disk-backed datasets (streaming, multi-worker prefetch, online

²Dataset instantiation: `torch.utils.data.TensorDataset` (datasets.py, lines 41–54).

³Dataset selection: `get_dataset("dino")` returns `dino_dataset` (datasets.py, lines 57–61).

⁴`DataLoader` creation: `DataLoader(dataset, batch_size=..., shuffle=True, drop_last=True)` (ddpm.py, lines 239–247).

transforms, pinned memory for GPU transfers). As a result, the optimization is well-suited for small in-memory datasets and for isolating training-loop overhead, but it may not transfer directly to realistic large-scale data pipelines.

6 Hyperparameter Configuration Recommendations

6.1 Recommended Configurations

Among the 64 evaluated configurations (Table 8), a single setup consistently emerges as the most effective across multiple criteria. The configuration **bs128_h256_l3_emb128_t100** achieves the lowest final loss (0.474) while maintaining a moderate training time of approximately 1.25 seconds per epoch (Table 9). This represents a 38% improvement in loss relative to the average configuration, while being 42% faster than the mean training time.

Table 8: Overall Benchmark Statistics

Metric	Value	Range
Mean Final Loss	0.761 ± 0.109	0.474 – 0.981
Relative Loss Variation	–	52%
Mean Training Time (s/epoch)	2.14 ± 1.35	0.76 – 4.70
Speedup Factor	–	6.2×
Success Rate	100%	–

Table 9: Top Performing Configurations

Configuration	Loss	Time (s)	BS	HS
<i>Best by Final Loss</i>				
bs128_h256_l3_emb128_t100	0.474	1.25	128	256
bs32_h128_l4_emb128_t50	0.527	4.50	32	128
bs32_h256_l3_emb64_t100	0.547	4.06	32	256
bs32_h256_l4_emb64_t50	0.563	4.70	32	256
bs32_h128_l3_emb64_t100	0.575	–	32	128
<i>Best by Training Speed</i>				
bs256_h128_l3_emb64_t50	0.793	0.76	256	128
bs256_h128_l3_emb64_t100	0.747	0.77	256	128
bs256_h128_l3_emb128_t50	0.792	0.77	256	128
bs256_h128_l3_emb128_t100	0.809	0.77	256	128
bs256_h256_l3_emb64_t50	0.796	0.77	256	256

For scenarios where rapid iteration is the primary objective, such as early-stage development or debugging, a faster configuration using a larger batch size (**bs256_h128_l3_emb64_t50**) reduces training time to 0.76 seconds per epoch. However, this speedup comes at the cost of significantly degraded convergence, with final loss values approximately 40% worse than the best-performing configuration.

Overall, the benchmark indicates that an intermediate batch size combined with sufficient model capacity offers the most favorable balance between computational efficiency and training quality.

6.2 Key Hyperparameter Effects

The batch size is the dominant factor controlling training speed (Table 10). Larger batches maximize throughput and minimize epoch duration, but consistently lead to poorer convergence. In contrast, small batches improve optimization dynamics but incur a substantial computational penalty. A batch size of 128 represents a robust compromise and should be preferred for most use cases. Notably, batch size 64 performs poorly across nearly all configurations and should generally be avoided.

Table 10: Batch Size Impact

Batch Size	Mean Loss	Std Loss	Min Loss	Time Mean (s)	Time Std (s)	Speedup vs. BS=32
32	0.726	0.129	0.527	4.30	0.32	1.0×
64	0.787	0.128	0.577	2.28	0.11	1.9×
128	0.770	0.106	0.474	1.29	0.06	3.3×
256	0.762	0.057	0.645	0.81	0.04	5.3×

Model capacity is best increased through width rather than depth (Tables 11 and 12). Increasing the hidden dimension from 128 to 256 improves convergence by 2.4%, whereas adding additional hidden layers (from 3 to 4) consistently degrades both loss (by 3.2%) and training speed (by 13.5%). This suggests that, for this low-dimensional diffusion task, deeper architectures introduce unnecessary complexity without providing meaningful representational benefits.

Table 11: Hidden Layer Size Impact

Hidden Size	Mean Loss	Std Loss	Min Loss	Time Mean (s)	Time Std (s)
128	0.770	0.107	0.527	2.08	1.32
256	0.752	0.111	0.474	2.19	1.39
Improvement	2.4%	–	–	-5.3%	–

Table 12: Network Depth Impact

Hidden Layers	Mean Loss	Std Loss	Min Loss	Time Mean (s)	Time Std (s)
3	0.749	0.103	0.474	1.98	1.22
4	0.773	0.115	0.527	2.29	1.46
Improvement	3.2%	–	–	13.5%	–

The number of diffusion timesteps also plays a critical role (Table 13). Increasing the number of timesteps from 50 to 100 improves final loss by approximately 4.3% on average, with a negligible impact on training time per epoch. This makes longer diffusion chains a high-impact, low-cost optimization. Similarly, embedding dimension (Table 14) shows a modest 1.9% improvement when doubled from 64 to 128, with minimal computational overhead.

Table 13: Diffusion Timesteps Impact

Timesteps	Mean Loss	Std Loss	Min Loss	Time Mean (s)	Time Std (s)
50	0.777	0.106	0.527	2.17	1.37
100	0.745	0.111	0.474	2.10	1.34
Improvement	4.3%	–	–	3.2%	–

Table 14: Embedding Dimension Impact

Embedding Dimension	Mean Loss	Std Loss	Min Loss	Time Mean (s)	Time Std (s)
64	0.768	0.111	0.547	2.11	1.35
128	0.754	0.108	0.474	2.16	1.36
Improvement	1.9%	–	–	-2.4%	–

Table 15 summarizes the relative importance of each hyperparameter based on their measured performance impact.

Table 15: Hyperparameter Importance Ranking

Hyperparameter	Performance Impact	Rank
Diffusion Timesteps	4.3% improvement (50 $\rightarrow$ 100)	1
Network Depth	3.2% improvement (4 $\rightarrow$ 3 layers)	2
Hidden Layer Size	2.4% improvement (128 $\rightarrow$ 256)	3
Embedding Dimension	1.9% improvement (64 $\rightarrow$ 128)	4
Batch Size	Non-monotonic	Complex

6.3 Architectural Design Principles

From these observations, several data-driven architectural principles can be derived. First, shallow networks with sufficient width outperform deeper alternatives, indicating that architectural simplicity is beneficial for both performance and efficiency. Second, diffusion process configuration—particularly the number of timesteps—has a larger impact on convergence than many architectural parameters. Finally, interactions between batch size and model capacity are non-trivial (Figure 4), reinforcing the need to tune these parameters jointly rather than in isolation.

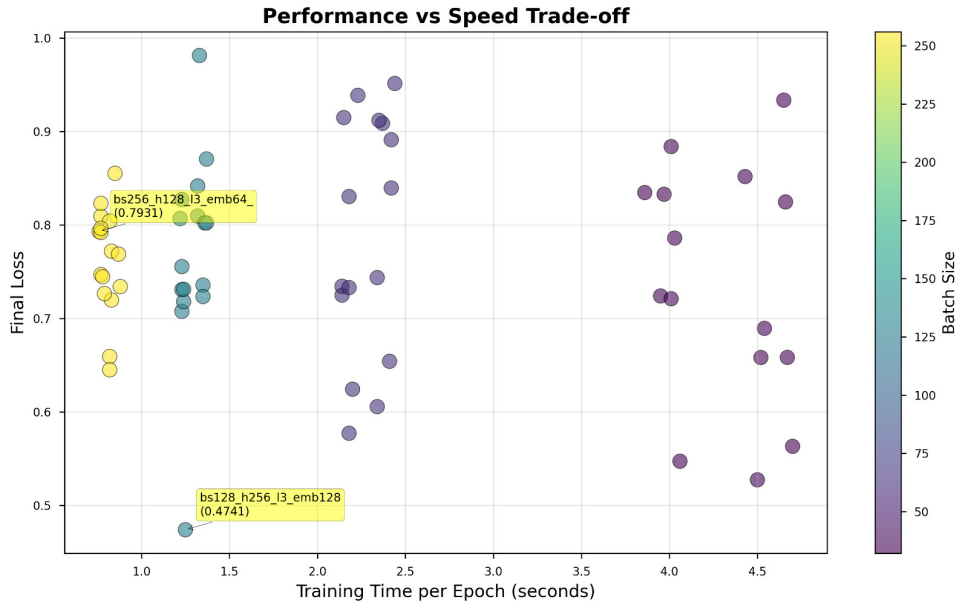


Figure 4: Batch-size Performance vs Speed Tradeoff

7 Training and Framework-Level Optimizations

Beyond hyperparameter selection, the profiler analysis highlights several opportunities for improving training efficiency. The largest limitation of the current implementation is the exclusive use of CPU resources, with no GPU acceleration. Migrating training to GPU is therefore the highest-priority optimization and is expected to yield speedups of one to two orders of magnitude.

At the framework level, a significant portion of training time is spent in PyTorch overhead, optimizer steps, and backward propagation. Techniques such as `torch.compile()`, mixed-precision training, and fused or optimized optimizers have the potential to reduce training time by an additional 20–40% once GPU execution is enabled.

Taken together, these results suggest a clear prioritization strategy. Immediate gains can be achieved by adopting the optimal hyperparameter configuration and enabling GPU training, which together provide the largest improvements in both speed and performance. Secondary optimizations, such as mixed precision and framework-level compilation, should be applied once GPU execution is in place.

Finally, the benchmark highlights several promising directions for future exploration, including larger batch sizes, longer diffusion chains, and wider network architectures. These extensions are expected to further refine the trade-off between computational efficiency and model quality, particularly in more demanding diffusion settings.

8 Conclusion

This study profiled a minimal DDPM training pipeline (*tiny-diffusion*) to understand where time and memory are spent in iterative diffusion learning. System-level benchmarking revealed substantial orchestration overhead in the end-to-end entrypoint: `full_train` is **38.3%** slower than the minimal `core_train` baseline. Python-level profiling confirmed that the dominant costs are the backward pass and the MLP forward (linear layers), with additional overhead tied to input construction (concatenations and positional embeddings). A targeted data-path change—batching directly from in-memory tensors instead of repeated DataLoader indexing—reduced wall-clock time by **16.9%** with only a negligible RSS increase (< 1 MiB), showing that interpreter-side input handling can be a significant bottleneck even in small models.

Hyperparameter benchmarking highlighted a clear quality–speed trade-off: the best overall configuration (`bs128_h256_l3_emb128_t100`) achieved the lowest final loss while keeping per-epoch time moderate. Across the explored space, wider and shallower networks tended to outperform deeper ones, and increasing diffusion timesteps ($50 \rightarrow 100$) improved convergence with minimal time impact.